ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		К.А. Кочин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

Работа с источником света и свойствами материала поверхности

по курсу: КОМПЬЮТЕРНАЯ ГРАФИКА

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4236		А.В. Панченко
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2023

1. Задание

Включить источник света, задать отражающие свойства поверхностей: диффузное, зеркальное отражение, цвет поверхности.

Вывести несколько объемных объектов. Каждый из них должен иметь различные свойства поверхностей (доминирующее зеркальное или диффузное отражение, различный цвет внешних поверхностей).

2. Текст программы

```
#include <iostream>
#include <glut.h>

// Угол поворота куба и пирамиды
float angleX = 0.0f;
float angleY = 0.0f;
int prevMouseX, prevMouseY;
bool isRotating = false;

enum ReflectionType {
    DIFFUSE,
    MIRROR,
    AMBIENT
};

void drawHouse(float roofColor[3], float wallColor[3], ReflectionType
reflectionType) {
    // Основание домика (квадрат)
    glBegin(GL_QUADS);
    glColor3fv(wallColor); // Цвет пола
    glVertex3f(-1.0f, -1.0f, -1.0f);
    glVertex3f(1.0f, -1.0f, -1.0f);
    glVertex3f(1.0f, -1.0f, 1.0f);
    glVertex3f(-1.0f, -1.0f, 1.0f);
    glEnd();

    // Стены домика (четыре стены)
    glBegin(GL_QUADS);
    GLfloat wallAmbient[] = { wallColor[0], wallColor[1], wallColor[2], 1.0f }; //
Амбиентный цвет для стен
    GLfloat wallDiffuse[] = { wallColor[0], wallColor[1], wallColor[2], 1.0f }; //
Диффузный цвет для стен
    GLfloat wallSpecular[] = { 0.0f, 0.0f, 0.0f, 1.0f }; // Зеркальный цвет для стен
    GLfloat wallShininess[] = { 0.0f }; // Степень отражения для стен

    if (reflectionType == MIRROR) {
        // Для зеркального отражения
        wallAmbient[3] = 0.0f; // Отключаем амбиентное отражение
        wallDiffuse[3] = 0.0f; // Отключаем диффузное отражение
        wallSpecular[0] = 1.0f;
        wallSpecular[1] = 1.0f;
        wallSpecular[2] = 1.0f;
        wallSpecular[3] = 1.0f;
        wallShininess[0] = 100.0f;
    }
    if (reflectionType == AMBIENT) {
        // Для смешанного отражения
        wallAmbient[3] = 1.0f; // Включаем амбиентное отражение
        wallDiffuse[3] = 0.0f; // Отключаем диффузное отражение
        wallSpecular[0] = 0.0f; // Отключаем зеркальное отражение
```

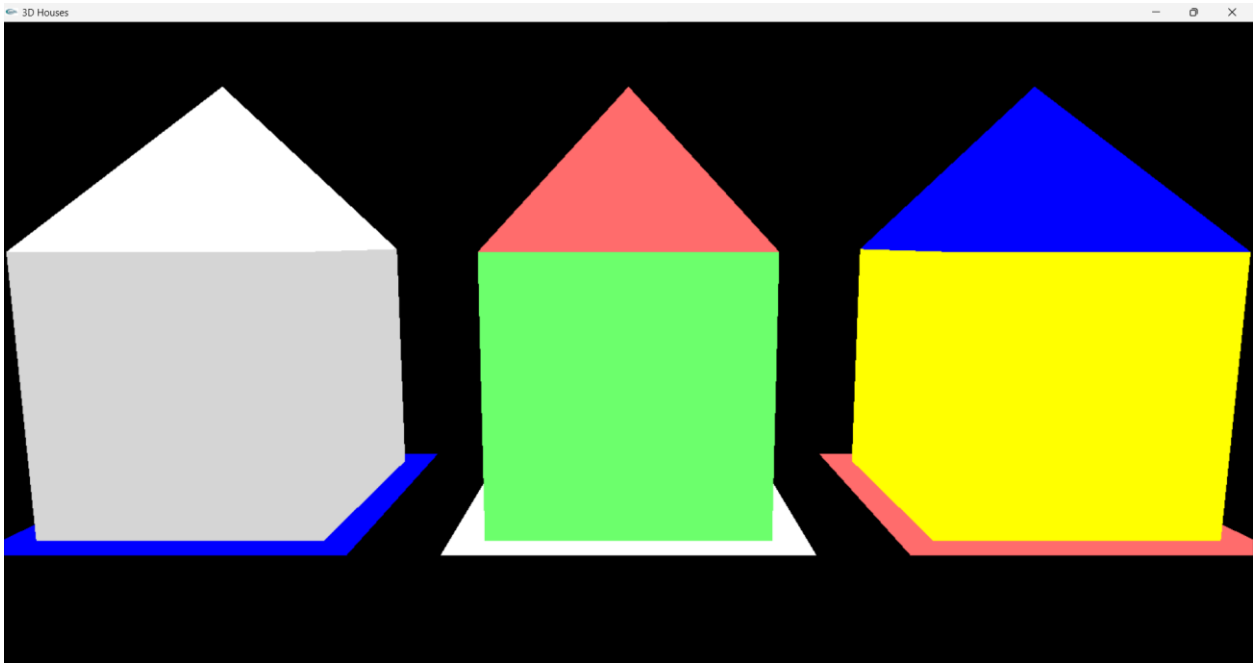
```
        wallSpecular[1] = 0.0f;
        wallSpecular[2] = 0.0f;
        wallSpecular[3] = 0.0f;
        wallShininess[0] = 0.0f; // Отключаем зеркальный блеск
    }
    else if (reflectionType == DIFFUSE) {
        wallAmbient[3] = 0.0f; // Отключаем амбиентное отражение
        wallDiffuse[3] = 1.0f; // Включаем диффузное отражение
        wallSpecular[0] = 0.0f; // Отключаем зеркальное отражение
        wallSpecular[1] = 0.0f;
        wallSpecular[2] = 0.0f;
        wallSpecular[3] = 0.0f;
        wallShininess[0] = 0.0f; // Отключаем зеркальный блеск
    }

    glMaterialfv(GL_FRONT_AND_BACK, GL_AMBIENT, wallAmbient);
    glMaterialfv(GL_FRONT_AND_BACK, GL_DIFFUSE, wallDiffuse);
    glMaterialfv(GL_FRONT_AND_BACK, GL_SPECULAR, wallSpecular);
    glMaterialfv(GL_FRONT_AND_BACK, GL_SHININESS, wallShininess);

    glColor3fv(wallColor); // Цвет стен
    glVertex3f(-0.8f, -1.0f, -0.8f);
    glVertex3f(0.8f, -1.0f, -0.8f);
    glVertex3f(0.8f, 0.6f, -0.8f);
    glVertex3f(-0.8f, 0.6f, -0.8f);

    glColor3fv(wallColor); // Цвет стен
    glVertex3f(-0.8f, -1.0f, 0.8f);
    glVertex3f(0.8f, -1.0f, 0.8f);
    glVertex3f(0.8f, 0.6f, 0.8f);
    glVertex3f(-0.8f, 0.6f, 0.8f);

    glColor3fv(wallColor); // Цвет стен
    glVertex3f(-0.8f, -1.0f, -0.8f);
    glVertex3f(-0.8f, -1.0f, 0.8f);
    glVertex3f(-0.8f, 0.6f, 0.8f);
    glVertex3f(-0.8f, 0.6f, -0.8f);

    glColor3fv(wallColor); // Цвет стен
    glVertex3f(0.8f, -1.0f, -0.8f);
    glVertex3f(0.8f, -1.0f, 0.8f);
    glVertex3f(0.8f, 0.6f, 0.8f);
    glVertex3f(0.8f, 0.6f, -0.8f);
    glEnd();

    // Крыша (пирамида)
    glBegin(GL_TRIANGLES);
    GLfloat roofAmbient[] = { roofColor[0], roofColor[1], roofColor[2], 1.0f }; //
Амбиентный цвет для крыши
    GLfloat roofDiffuse[] = { roofColor[0], roofColor[1], roofColor[2], 1.0f }; //
Диффузный цвет для крыши
    GLfloat roofSpecular[] = { 0.0f, 0.0f, 0.0f, 1.0f }; // Зеркальный цвет для
крыши
    GLfloat roofShininess[] = { 0.0f }; // Степень отражения для крыши

    if (reflectionType == MIRROR) {
        // Для зеркального отражения
        roofAmbient[3] = 0.0f; // Отключаем амбиентное отражение
        roofDiffuse[3] = 0.0f; // Отключаем диффузное отражение
        roofSpecular[0] = 1.0f;
        roofSpecular[1] = 1.0f;
        roofSpecular[2] = 1.0f;
        roofSpecular[3] = 1.0f;
        roofShininess[0] = 100.0f;
    }
```

```
    if (reflectionType == AMBIENT) {
        // Для смешанного отражения
        roofAmbient[3] = 1.0f; // Включаем амбиентное отражение
        roofDiffuse[3] = 0.0f; // Отключаем диффузное отражение
        roofSpecular[0] = 0.0f; // Отключаем зеркальное отражение
        roofSpecular[1] = 0.0f;
        roofSpecular[2] = 0.0f;
        roofSpecular[3] = 0.0f;
        roofShininess[0] = 0.0f; // Отключаем зеркальный блеск
    }
    else if (reflectionType == DIFFUSE) {
        roofAmbient[3] = 0.0f; // Отключаем амбиентное отражение
        roofDiffuse[3] = 1.0f; // Включаем диффузное отражение
        roofSpecular[0] = 0.0f; // Отключаем зеркальное отражение
        roofSpecular[1] = 0.0f;
        roofSpecular[2] = 0.0f;
        roofSpecular[3] = 0.0f;
        roofShininess[0] = 0.0f; // Отключаем зеркальный блеск
    }

    glMaterialfv(GL_FRONT_AND_BACK, GL_AMBIENT, roofAmbient);
    glMaterialfv(GL_FRONT_AND_BACK, GL_DIFFUSE, roofDiffuse);
    glMaterialfv(GL_FRONT_AND_BACK, GL_SPECULAR, roofSpecular);
    glMaterialfv(GL_FRONT_AND_BACK, GL_SHININESS, roofShininess);

    glColor3fv(roofColor); // Цвет крыши
    glVertex3f(-0.8f, 0.6f, -0.8f);
    glVertex3f(0.8f, 0.6f, -0.8f);
    glVertex3f(0.0f, 1.6f, 0.0f);

    glColor3fv(roofColor); // Цвет крыши
    glVertex3f(-0.8f, 0.6f, 0.8f);
    glVertex3f(0.8f, 0.6f, 0.8f);
    glVertex3f(0.0f, 1.6f, 0.0f);

    glColor3fv(roofColor); // Цвет крыши
    glVertex3f(-0.8f, 0.6f, -0.8f);
    glVertex3f(0.0f, 1.6f, 0.0f);
    glVertex3f(-0.8f, 0.6f, 0.8f);

    glColor3fv(roofColor); // Цвет крыши
    glVertex3f(0.8f, 0.6f, -0.8f);
    glVertex3f(0.0f, 1.6f, 0.0f);
    glVertex3f(0.8f, 0.6f, 0.8f);
    glEnd();
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    glTranslatef(0.0f, 0.0f, -5.0f);
    glRotatef(angleX, 1.0f, 0.0f, 0.0f);
    glRotatef(angleY, 0.0f, 1.0f, 0.0f);

    // Включаем освещение
    glEnable(GL_LIGHTING);
    glEnable(GL_LIGHT0);

    // Устанавливаем глобальные параметры фонового освещения
    GLfloat ambientLight[] = { 0.2f, 0.2f, 0.2f, 1.0f }; // Измените значения, если
необходимо
    glLightfv(GL_LIGHT0, GL_AMBIENT, ambientLight);

    // Отрисовываем первый домик
    float roofColor1[] = { 1.0f, 0.0f, 0.0f }; // Красная крыша
```

```
    float wallColor1[] = { 0.0f, 1.0f, 0.0f }; // Зеленые стены
    drawHouse(roofColor1, wallColor1, MIRROR);

    // Перемещаемся к новой позиции для второго домика
    glTranslatef(2.5f, 0.0f, 0.0f);

    // Отрисовываем второй домик
    float roofColor2[] = { 0.0f, 0.0f, 1.0f }; // Синяя крыша
    float wallColor2[] = { 1.0f, 1.0f, 0.0f }; // Желтые стены
    drawHouse(roofColor2, wallColor2, DIFFUSE);

    // Перемещаемся к новой позиции для третьего домика
    glTranslatef(-5.0f, 0.0f, 0.0f);

    // Отрисовываем третий домик
    float roofColor3[] = { 1.0f, 1.0f, 1.0f }; // Белая крыша
    float wallColor3[] = { 0.6f, 0.6f, 0.6f }; // Серые стены
    drawHouse(roofColor3, wallColor3, AMBIENT);

    glDisable(GL_LIGHTING);

    glFlush();
    glutSwapBuffers();
}

void reshape(int width, int height) {
    glViewport(0, 0, width, height);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(45.0f, (float)width / (float)height, 0.1f, 100.0f);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

void mouse(int button, int state, int x, int y) {
    if (button == GLUT_LEFT_BUTTON) {
        if (state == GLUT_DOWN) {
            isRotating = true;
            prevMouseX = x;
            prevMouseY = y;
        }
        else if (state == GLUT_UP) {
            isRotating = false;
        }
    }
}

void motion(int x, int y) {
    if (isRotating) {
        int deltaX = x - prevMouseX;
        int deltaY = y - prevMouseY;
        angleX += static_cast<float>(deltaY) * 0.5f;
        angleY += static_cast<float>(deltaX) * 0.5f;
        prevMouseX = x;
        prevMouseY = y;
        glutPostRedisplay();
    }
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(800, 600);
```

```
    glutCreateWindow("3D Houses");

    glutDisplayFunc(display);
    glutReshapeFunc(reshape);
    glutMouseFunc(mouse);
    glutMotionFunc(motion);

    glEnable(GL_DEPTH_TEST);

    glutMainLoop();

    return 0;
}
```

3. Скриншоты выполнения работы



4. Вывод

Данный код представляет собой пример приложения, использующего библиотеку OpenGL для отображения трех 3D домиков с различными типами отражения (DIFFUSE, MIRROR, AMBIENT). Пользователь может вращать сцену с помощью мыши, а каждый домик состоит из куба (стен) и пирамиды (крыши), на которые наложены различные свойства отражения света, создавая эффекты зеркальности, диффузного и смешанного отражения. Этот пример демонстрирует основные концепции работы с OpenGL, такие как настройка освещения, настройка материалов, управление матрицами преобразований и обработка пользовательского ввода.